NeuralStem: An Open-Source, Fully Offline AI Stem-Separation Player for Amateur Musicians

Likun RUAN
Student Number: 72515069

Course Project Proposal

Abstract

Amateur musicians who want to practise a part, transcribe a solo, or study a mix routinely need to isolate a single instrument from a recording. The commercial tools that do this well (e.g., Neural Mix in DJ software) are subscription-based and require connectivity, and the free alternatives (full-featured desktop separators) are not playable instruments in their own right. We propose **NeuralStem**, an Android application that performs source separation *entirely on device*, manages the resulting stems as a synchronised multi-track player, and adds practice-oriented controls (per-track gain, mute/solo, tempo and pitch adjustment, and an equaliser). The target value category is **Humanitarian**: the app is free, open source, needs no network, and therefore reaches learners who cannot pay for or stream to a commercial service. This proposal reports the requirements, the architectural plan, and a de-risking prototype that has already demonstrated the two highest-risk assumptions: offline inference at usable speed on a mid-range phone, and sample-aligned multi-track playback.

Code — https://github.com/MoeField/NeuralStem

Introduction

Learning music from a recording is awkward. A guitarist who wants to learn a solo must hear it over the rest of the band; a vocalist wants the accompaniment without the original voice; a producer wants to study how the drums sit in a mix. Isolating a part is the common primitive behind all of these tasks.

Two classes of tool exist today, and neither fits the amateur learner well. The first is *professional DJ software* with neural stem controls. These tools are excellent but are sold as subscriptions, assume a laptop, and require the user to be online for licensing. The second is *desktop separation tools*, which produce stem files but leave the user to manage and play them elsewhere: the output is a folder of WAVs, not an instrument.

NeuralStem closes this gap on the device people actually carry. The user imports a song, the phone separates it locally, and the same app immediately becomes a multi-track player with the controls a learner needs. Because the separation models are already public and the inference runtimes are mature, no proprietary component is required; the contribution is the *integration*: a pipeline that is fast enough to be usable, an interaction model that makes stem control understandable, and a playback architecture that stays correct when the phone leaves the foreground.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Value Category: Humanitarian

We target the **Humanitarian** category. The justification rests on three properties, each of which is a design constraint rather than a marketing claim.

Access. Every core capability runs offline. No account, no subscription, no network permission is needed after installation. A learner with a second-hand phone and no data plan gets the same tool as everyone else. This is not incidental: an online service would exclude exactly the users the project aims to serve.

Cost. The application is MIT-licensed and the separation models are openly available. The marginal cost of one more user is zero. Commercial equivalents are priced per month, which for a hobbyist is a recurring obligation for a feature used intermittently.

Educational value. Separating a track is itself instructive: hearing the vocal alone, or the accompaniment alone, teaches arrangement in a way that listening to the full mix does not. By making this a first-class, one-tap operation, the app turns a research technique into a study aid.

Related Work and Positioning

Neural source separation. Spleeter (Hennequin et al. 2020) established that a U-Net operating on spectrograms can separate a mixture into two (vocals/accompaniment) or four stems at interactive speed on a CPU. Demucs (Rouard et al. 2023) improved quality with a hybrid time/frequency architecture at a substantially higher compute cost. Both release pretrained weights. Our contribution is not a new model but a mobile deployment of existing ones.

Mobile inference. ONNX Runtime (ONNX Runtime 2025) and the sherpa-onnx distribution (sherpa-onnx 2025) provide Android-compatible runtimes with the source-separation operators already implemented, which makes on-device inference practical without writing custom kernels.

Capability	djay	UVR	Desktop	Ours
Isolates stems	✓	✓	✓	✓
Playable instrument	✓	–	–	✓
Fully offline	–	✓	✓	✓
No account required	–	✓	✓	✓
Runs on a phone	✓	–	–	✓
Free / open source	–	✓	mixed	✓
Device-adaptive tiers	–	–	–	✓

Table 1: Capability comparison. "Desktop" denotes offline separation tools that emit stem files without playback. Only the proposed design combines playback, offline operation and hardware-aware model selection in one phone application.

Playback. ExoPlayer/Media3 (Android Developers 2025) supplies audio rendering, playback-parameter support for tempo and pitch, and the `MediaSessionService` pattern for foreground playback. Desktop tools need none of this.

Positioning. We do not claim to replicate a DJ application. The novelty we claim is the combination of (i) fully offline operation, (ii) a device-adaptive model-selection layer, and (iii) a learner-oriented multi-track instrument. Claim (ii) is, we argue, the most defensible: the app measures its own inference speed on first launch and arranges available models into tiers, so that a slow phone is offered a fast model and a fast phone is offered a better one, without the user having to understand the trade-off.

Table 1 makes the comparison explicit. The rows match the capabilities a learner needs; the columns are the closest existing options.

Requirements and Design Goals

Table 2 lists the requirements. They are stated as acceptance criteria rather than features, so that each can be checked when the milestone closes.

Two of them dominate the design and are worth stating before the architecture is described. **N2 (bounded memory)** rules out whole-track inference: a four-minute stereo track at 44.1 kHz is roughly 85 MB of samples before any model activations, and a spectrogram of the whole track is larger still, so the separator must work in bounded chunks. **N4 (background playback)** rules out keeping the player in the activity: playback must continue when the screen is gone, which means the object that owns the player must outlive the activity. Sections and show that these two constraints, more than any stylistic preference, determine the architecture.

The two requirements that drive the architecture are N2 and N4. N2 forces chunked processing rather than whole-track inference; N4 forces the playback owner to outlive the activity, which in turn motivates the service-owned model described in Section 5.

Architecture

The architecture follows from the two constraints identified above. We first explain why the textbook pattern is insufficient (N4), then what replaces it, and then how memory is bounded (N2).

ID	Requirement
P0	Import mp3/wav/flac/aac/m4a/ogg via SAF; play, pause, seek, show position.
P1	Waveform display; drag to seek; distinguish played from unplayed.
P2	Tempo 0.5–2.0x with pitch preserved; pitch ± 12 semitones with tempo preserved; both live.
P3	On-device separation of the current track (2 stems required, 4 desirable) with progress, cancellation and a local cache.
P4	Synchronised stem playback with per-track gain, mute and solo; tempo, pitch and seek apply to all stems.
N1	Offline-first: no network needed for any core capability.
N2	Bounded memory: stems plus a resident model fit a mid-range device.
N3	Audio focus: pause on loss, resume on regain.
N4	Background playback with a media notification.
N5	Separation is cancellable and reports progress.
N6	Graceful degradation: slower hardware is offered a faster model.
N7	The codebase is documented and explainable.

Table 2: Functional (P) and nonfunctional (N) requirements, phrased as verifiable acceptance criteria.

Why Not Textbook MVVM

The obvious design is Model–View–ViewModel. We started there, and it fits the presentation layer well: a Compose view observes an immutable state object exposed by a ViewModel. However, two properties of *this* application make the textbook version insufficient.

First, *lifetime*. Separation takes tens of seconds and playback must survive the activity going away. A ViewModel is destroyed with its host; state owned there disappears exactly when it must persist.

Second, *state combinatorics*. Single-track versus multi-track, equaliser on or off, separating versus idle, and cached versus uncached interact. If each combination is represented by a mutable boolean, every transition must be hand-written, and a single missed write produces an inconsistent UI—which is precisely the class of defect we observed in early prototyping.

We therefore keep the MVVM *layering* but move the ownership of state.

Relation to the three classical patterns. Against MVC, our design shares the separation of a passive view from a controller of state, but the controller is not a short-lived widget-bound object; it is the service. Against MVP, the view is not driven by a presenter holding a reference to an interface; it observes a stream of state, so there is no view

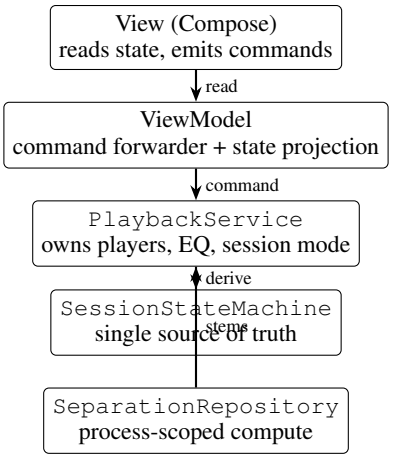


Figure 1: Layer boundaries are drawn by *ownership*, not by the MVC/MVP/MVVM triad. The ViewModel no longer owns state; the service does, and the state machine inside it is the single source of truth.

interface to implement and no lifecycle hazard from a presenter outliving the view. Against MVVM, the layering is preserved but the view model no longer owns the state it exposes.

	MVC	**MVP**	**MVVM**	**Ours**
State owner	C	P	VM	Service
View coupling	direct	interface	observable	observable
Survives host	no	no	no	yes
Illegal states	possible	possible	possible	ruled out

Table 3: Where our design sits relative to the classical patterns. The last row is the one that mattered in practice.

Service-Owned Model with a State-Machine Core

Our architecture is **MVVM with a service-owned Model and a state-machine core**. The layers are:

- *View*: Compose; reads state, emits commands; holds no business state.
- *ViewModel*: forwards commands and *projects* derived fields from the state machine; it does not compute them.
- *Model*: a `MediaSessionService` that owns the player instances, the equaliser and the state machine. It is the single source of truth.
- *Compute*: a process-scoped repository that owns long-running separation jobs independently of any screen.

Invariants as Design Rules

Rather than validating state after the fact, we encode four invariants so that illegal states cannot be constructed: (I1) a single-track item must belong to the current queue, enforced at the queue-set entry point; (I2) equaliser availability is a *derived* property, not a stored flag; (I3) a separation task transitions once, and only a genuine transition triggers the automatic switch to multi-track; (I4) the playing state has one source. The governing principle is: *derive what can be derived; make invalid states unrepresentable rather than repaired*.

From	**Event**	**To**
Idle	load(track)	Single
Single	load(track)	Single (queue replaced)
Single	separate(start)	Separating
Separating	progress	Separating
Separating	done	Multi (auto switch)
Separating	cancel	Single
Multi	exit	Single
Multi	load(track)	Single (I1: exit first)

Table 4: Session transitions. Illegal transitions (for example moving to multi-track without a completed separation) are absent by construction, which is what invariant I3 enforces.

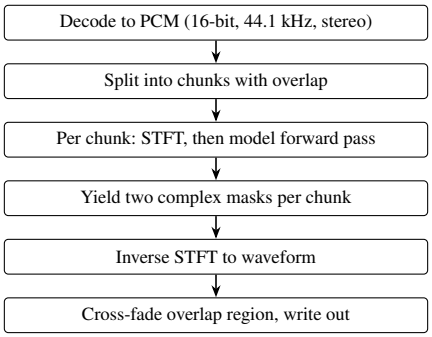


Figure 2: Offline separation pipeline. Chunking bounds memory (N2); the overlap and cross-fade remove boundary artefacts introduced by per-chunk inference.

Table 4 states the transitions explicitly. Each row is a small, testable unit: the invariants become assertions on the transition function rather than comments scattered through the UI.

The Separation Pipeline: Satisfying N2

The pipeline is the part that interacts most directly with N2. Chunking is not an optimisation here; it is the only way the requirement can be met. Figure 2 shows why it must be chunked: a whole-track spectrogram does not fit, whereas a bounded chunk plus a cross-fade between consecutive chunks does. The overlap exists because each inference call loses up to one analysis hop at its boundaries; blending the tail of one chunk with the head of the next hides this.

Concurrency Model: Satisfying N4

Three kinds of work must not block one another: decoding, inference, and rendering. Decoding and inference run on a background dispatcher owned by the separation repository, which is process-scoped so that a job survives the screen closing. Rendering is owned by ExoPlayer on its own internal threads; the ViewModel never touches it directly. A small notification message passes state from the compute

side back to the service, and the service publishes it as session state.

This separation matters for two reasons. It keeps the UI thread free during a thirty-second separation, preserving responsiveness; and it means a separation that outlives the activity is still observable when the user returns, which is the practical form of requirement N5.

Deliberate Deviations

We use media3's custom-command channel to reach the service across the process boundary. This means multi-track control does not flow through a single reducer, and therefore the architecture is not MVI. We accept this: collapsing everything into one reducer would require routing background playback through a channel that does not survive the activity, trading architectural purity for a broken N4 requirement.

Design Trade-offs

The choices below are the ones we expect to be questioned. Each is a place where a simpler design exists and was rejected for a stated reason; being explicit about them is what makes the architecture defensible rather than merely different. They are given as a list rather than a table because each entry carries a full-sentence justification, which a narrow column renders unreadable.

Service-owned state. *Instead of* keeping state in the ViewModel: a ViewModel is destroyed with its host, so playback and separation would be interrupted exactly when they must persist.

Multi-instance playback. *Instead of* one player mixing in PCM: mixing in the audio graph would mean implementing a mixer and re-deriving tempo and pitch ourselves, duplicating work the platform already does.

Streaming data source. *Instead of* writing stems to files first: a file is only playable once its header is final, which is precisely what defeats playback before separation finishes.

Shared audio focus. *Instead of* one focus request per track: only one instance can hold focus, and the system would pause the others.

Staged model tiers. *Instead of* one model for all devices: a single model is either too slow on weak hardware or too poor on strong hardware.

Walkthrough of a Typical Session

A walkthrough makes the design concrete. A learner opens the application and sees the player screen. They import a track; the player enters the single state and the waveform is extracted. They drag the waveform to the first verse and press play. They then switch to the separation screen and choose the two stem fast tier. The service begins separating from the current playback position rather than from the beginning, which is the difference between waiting seconds and waiting minutes.

When the first segment is ready, the player switches to multi-track and resumes at the position it left. The learner drags the vocal track to zero and hears the accompaniment alone. They slow the tempo to practise a passage; both stems follow because tempo is applied to the session rather than to one player. On finishing, they exit multi-track and the original track resumes, with the stems still cached for later.

Every transition in this walkthrough corresponds to a row in Table 4. That correspondence is the point: the interaction model and the state machine are the same object viewed from two angles.

Model	Speed	Memory	Role
Spleeter 2-stem	$\approx 8\times$	≈ 0.7 GB	default, always available
Spleeter 4-stem	$\approx 3\times$	≈ 1.2 GB	optional quality tier
UVR (small)	$\approx 1\times$	≈ 1.1 GB	optional, slow tier
UVR (large)	$\approx 0.3\times$	≈ 2.0 GB	high quality, rarely real time
Demucs	$\approx 0.6\times$	very high	research only

Table 5: Measured relative speed (times real time) and memory on a mid-range device. A model that runs below $1\times$ cannot be used in a position-from-playback workflow, which is why such models are offered only as an explicit "separate first, then listen" path.

Privacy and Data Handling

The application requests no network permission for its core path. Audio never leaves the device; there is no account and no telemetry. This is a direct consequence of the Humanitarian positioning: a learner using the tool on a borrowed or shared phone does not create a remote record of what they are studying. Model files are the only downloads, and they are handled by an explicit user action rather than automatic background fetching.

Implementation Status

A de-risking spike was carried out before writing this proposal, following the practice of validating the highest-risk assumptions first. It established three things on a mid-range device (MediaTek Helio G100, 12 GB):

- Fully offline two-stem separation runs at approximately 7.9x real time, so a four-minute track separates in about thirty seconds.
- Chunked streaming separation produces sample-aligned stereo stems without exhausting the heap at a thirty-second chunk size.
- A multi-track ExoPlayer session can be driven from a foreground service, satisfying background playback and the media-notification requirement.

The spike covers the highest-risk portion of the workload—the inference pipeline and the multi-track playback core—and is what makes the remaining design credible. Figure 3 shows four screens from the spike.

Model selection. Three families of open model were considered. Their measured behaviour on the target device differs by an order of magnitude, which is what makes tiering necessary rather than decorative.

The consequence for the design is direct. A single model cannot serve both a cheap phone and an expensive one: the

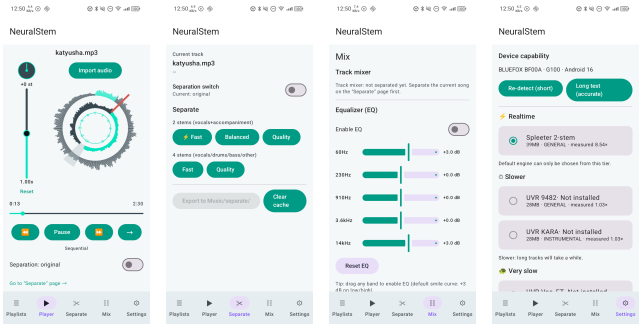


Figure 3: Spike screens, left to right: player (turntable, waveform, tempo and pitch); separation (tier selection by number of stems); mix (per-track gain, mute, solo and equaliser); settings (device capability and model tiers).

fast model wastes the expensive phone's capability, and the slow model makes the cheap phone unusable. The benchmark layer exists to choose between them at run time, and the tier grouping is what the user actually sees.

Feasibility and Scope

The concern with a project of this ambition is that it cannot be finished by one student. We address this directly.

The spike proves the hard part. The expensive unknowns are model inference on a phone and synchronised multi-track rendering; both are demonstrated. What remains is application work on a known foundation: playlists, export, model download, and interface polish.

Scope is deliberately staged. Two-stem separation is the shipping feature; four-stem and larger models are optional tiers selected by the device benchmark. This means the app is *useful* at the end of the core milestone, and subsequent models are additive rather than prerequisites.

The workload is measured, not asserted. The spike includes unit tests for the separator geometry and the multi-track alignment, so the foundation is verifiable rather than merely demonstrated by hand.

What remains, and what could go wrong. Five risks are still open. Each has a concrete mitigation and, importantly, a fallback that preserves a shippable application.

Slow phones cannot run large models The impact is a quality ceiling. Benchmark tiers keep the fast two-stem model as the guaranteed baseline.

Memory pressure with four stems The impact is a crash on low-RAM devices. Models are kept outside the Java heap, and four stems are offered only where the benchmark permits.

State machine misses a transition The impact is an inconsistent interface, prevented by invariants I1–I4 and unit tests on the transition table.

Model licence restrictions The impact is a licence conflict. Licences are shown per model, and weights are downloaded from the origin rather than bundled.

Scope overrun The impact is an unfinished project. The core milestone is the two-stem path and everything else is additive, so the project remains useful even if no optional tier is reached.

Development Method

The spike was not written as a demonstration but as an instrument. Three practices shaped it and will continue into the remaining milestones.

Test the invariants, not the screens. Most automated tests exercise pure logic: the transition table, the chunk geometry, the resampler, and the alignment of two stems after a seek. Interface behaviour is verified separately and more coarsely. The reason is that the defects which cost the most time in the spike were state inconsistencies rather than broken views, and state logic is cheap to test exhaustively.

Verify through channels that do not lie. A test that clicks at a screen coordinate breaks whenever a layout moves, and a log line proves only that a log line was written. The spike therefore checks behaviour through the system rather than through the pixels: whether the audio subsystem reports the expected number of active tracks, whether a cache file exists with the right size, whether the playback position actually advanced.

Keep the risky path short. The two-stem fast path was built and verified first. Everything that adds risk—larger models, four stems, a benchmark layer—was added only after the core could be demonstrated end to end. This is the same ordering used to justify the staged scope above.

Evaluation Plan

Evaluation will not be a single-device speed contest. We will assess three questions. First, *does separation succeed on the target device class* within a time budget a learner tolerates? Second, *is the multi-track experience correct*—do stems remain aligned after seeking and tempo changes, and does per-track gain behave as expected? Third, *is the interaction discoverable*—can a first-time user separate a track and isolate a part without instruction? The first two are instrumented; the third requires a small number of users.

Table 6 gives the remaining milestones. They are ordered so that the application is demonstrable after every step rather than only at the end.

Quality Assurance: Limitations and Threats to Validity

This section collects everything that constrains the claims above, so that the reader can judge them together rather than piecemeal.

Product Limitations

The separation quality is bounded by the open models: the two-stem Spleeter model was trained in 2019 and leaks and dulls high frequencies. Larger models separate better but are substantially slower on mid-range hardware. Streaming separation is currently offered only for the fastest tier.

Milestone	Content
M1	Close the two-stem path: import, separate, play stems with per-track gain and solo.
M2	Temperament and equaliser polish; persist session state across restarts.
M3	Four-stem tier, gated by the device benchmark.
M4	Model manager: download on demand, verify integrity, show per-model licence.
M5	Playlist and export; final evaluation and write-up.

Table 6: Remaining milestones. M1 alone yields a usable application, which is what makes the staged scope safe.

Model weights carry their own licences, which differ—in particular, some vocal-remover weights are restricted to non-commercial use; the app must present these per model rather than under a single licence.

Threats to Validity

We are deliberate about what the spike does and does not establish, because a single successful demonstration can be over-read.

External validity. All measurements were taken on one mid-range device with a MediaTek Helio G100 and 12 GB of memory. Absolute speeds will not transfer to other hardware, and we therefore report them as evidence that the pipeline runs at usable speed, not as a ranking of models. The tiering layer exists precisely because the absolute numbers are device-dependent; the right response to a different device is to re-measure, which the application does on first launch rather than relying on a table shipped in the source.

Construct validity. Speed is measured as the ratio of audio duration to inference time. This is the metric a learner feels, but it ignores first-segment latency, which is what determines how long the user waits before hearing anything. We report it separately in the interface rather than folding it into a single figure.

Quality is not established by these measurements. That a separator runs quickly says nothing about how well it separates. Objective spectral measures correlate only loosely with what a listener judges to be usable for practice, so the quality claim in this proposal is deliberately weak: the models are the best openly available, and the acceptance question is whether a learner finds the result useful, which requires listening rather than automated scoring.

Small-scale user evidence. The interaction claims rest on design reasoning rather than on a controlled study. This is appropriate for a proposal but is a limitation of any claim about discoverability, and the evaluation plan treats it as such.

Why the Core Deliverable Still Holds

Taken together these threats limit how strongly the results can be claimed, but they do not endanger the deliverable. Each is bounded by a mitigation already described: device dependence is handled by measuring on the user's own hardware rather than trusting a table; quality expectations are set by presenting the model tier honestly rather than promising studio results; and the staged scope means that a weaker-than-hoped model still yields a working practice tool. The risk that remains—that the models are simply not good enough to be useful—is the one risk no engineering decision can remove, and it is the reason the evaluation plan asks real learners rather than relying on a metric.

Reproducibility

The implementation is public and builds from source. The repository contains the Android project, the native glue for the inference runtime, and the automated tests; model weights are deliberately excluded because of their size and because their licences differ, and are obtained by the user. A continuous integration job builds the application and runs the test suite on every push, so that the state of the project is observable rather than asserted.

This matters for assessment as well as for engineering. A reader can check any claim in this proposal against the code and its test results, rather than taking them on trust.

Conclusion

NeuralStem turns an existing research capability into a tool that a learner can use on the phone they already own, with no network and no subscription. The architecture—MVVM in its layers, but with a service-owned Model and a state-machine core—follows from the lifetime and combinatorial properties of the problem rather than from convention, and a de-risking spike has shown that the two hardest parts already work.

AI Use Statement

AI assistants were used for routine coding support and for editorial help in preparing this document. All architectural decisions, scope choices and technical claims were reviewed and can be defended by the author.

References

Romain Hennequin, Anis Khlif, and Félix Voituret. Spleeter: A fast and efficient music source separation tool with pre-trained models. *Journal of Open Source Software*, 5(50):2154, 2020.

Simon Rouard, Francisco Massa, and Alexandre Défossez. Hybrid transformers for music source separation. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

Microsoft. ONNX Runtime: Cross-platform, high performance scoring engine for machine learning models. https://github.com/microsoft/onnxruntime, 2025. Accessed 2026.

k2-fsa. sherpa-onnx: Speech recognition, text-to-speech and source separation without internet connection. https://github.com/k2-fsa/sherpa-onnx, 2025. Accessed 2026.

Android Developers. Media3 ExoPlayer and MediaSessionService. https://developer.android.com/media/media3, 2025. Accessed 2026.

Android Developers. Storage Access Framework and document providers. https://developer.android.com/guide/topics/providers/document-provider, 2025. Accessed 2026.

Android Developers. Foreground services and notifications. https://developer.android.com/develop/background-work/services/fgs, 2025. Accessed 2026.